

Reviewer Pack — Minimal Index of Automorphism Groups in Quiver Representatio...

Entry 6551baf88517 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 14:32:49 UTC

Minimal Index of Automorphism Groups in Quiver Representations Bounds Frege Proof Depth

Entry ID: 6551baf88517 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 14:32:49 UTC

1. Conjecture

Field A (mathematical branch): Quiver Representations in Algebraic Geometry **Field B** (complexity object): Boolean Circuit Complexity (specifically, Frege proof depth)

Statement:

For every d -regular graph G , the minimal index of automorphism groups ($m_index(G)$) in its associated quiver representation φ_G is linearly correlated with its Frege proof depth $w_F(\varphi_G)$, such that $m_index(G) = O(w_F(\varphi_G))$.

Rationale (proposer's reasoning):

Quiver representations provide a categorical framework for studying algebraic structures, and their automorphism groups can reveal underlying symmetries. If the minimal index of these groups bounds the Frege proof depth, it could imply a connection between algebraic structure and circuit complexity, potentially leading to new insights into proof systems.

Taxonomy category: AlgebraicGeometry × CircuitComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c1056832fdccebe2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that minimal index of automorphism groups in quiver representations bounds Frege proof depth, support is indicated by a Spearman's rank correlation coefficient (ρ) ≥ 0.7 over 30 random graphs G with $n \leq 40$ variables and $m_index(G) = O(w_F(\varphi_G))$. Falsification occurs if $\rho < 0.5$ or any seed produces $m_index(G) > 2 * w_F(\varphi_G)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quiver representations" AND "automorphism groups" AND "Boolean circuit complexity" - "Frege proof depth" IN "algebraic geometry" - "minimal index of automorphism groups" related TO "quiver representations" AND "Boolean circuit complexity"

Top relevant hits considered: - [http://arxiv.org/abs/2204.10334v1] Machine Learning Algebraic Geometry for Physics - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/math/0502387v3] Multiplier ideals in algebraic geometry - [http://arxiv.org/abs/cs/0608100v1] Similarity of Semantic Relations - [http://arxiv.org/abs/0810.1207v1] A Layered Grammar Model: Using Tree-Adjoining Grammars to Build a Common Syntactic Kernel for Related Dialects - [http://arxiv.org/abs/1310.8154v3] Characteristic cohomology of the infinitesimal period relation - [s2:2509.16824] AC⁰[p]-Frege Cannot Efficiently Prove that Constant-Depth Algebraic Circuit Lower Bounds are Hard - [s2:10.48550/arXiv.2509.16824] AC⁰[p]-Frege Cannot Efficiently Prove that Constant-Depth Algebraic Circuit Lower Bounds are Hard

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_d_regular_graph(n, d):
    if d * n % 2 != 0:
        return None
    degree = d * n // 2
    adj_list = [[] for _ in range(n)]
    edges_added = set()

    def add_edge(u, v):
        if (u, v) not in edges_added and (v, u) not in edges_added:
            adj_list[u].append(v)
            adj_list[v].append(u)
            edges_added.add((u, v))
            edges_added.add((v, u))

    for i in range(n):
        for j in range(i + 1, n):
            if len(adj_list[i]) < degree and len(adj_list[j]) < degree:
                add_edge(i, j)
```

```

    return adj_list

def is_valid_quiver_representation(graph):
    if graph is None:
        return False
    n = len(graph)
    for i in range(n):
        if len(graph[i]) != d:
            return False
    return True

def compute_minimal_index_of_automorphism_groups(adj_list):
    n = len(adj_list)
    visited = [False] * n

    def dfs(node, parent):
        stack = [(node, parent)]
        while stack:
            current, p = stack.pop()
            if not visited[current]:
                visited[current] = True
                for neighbor in adj_list[current]:
                    if neighbor != p:
                        stack.append((neighbor, current))

    connected_components = 0
    for i in range(n):
        if not visited[i]:
            dfs(i, -1)
            connected_components += 1

    return n // connected_components

def construct_frege_proof(adj_list):
    n = len(adj_list)
    proof_steps = []
    for i in range(n):
        for j in adj_list[i]:
            proof_steps.append((i, j))
    return proof_steps

def compute_frege_proof_depth(proof_steps):
    depth = 0
    max_depth = 0
    stack = []

    for step in proof_steps:
        u, v = step
        while stack and stack[-1][1] != u:
            stack.pop()
        if not stack:
            depth += 1
        stack.append((u, v))
        max_depth = max(max_depth, depth)

    return max_depth

```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    metric_name = "Spearman's rank correlation"
    total_metric_value = 0
    instances_tested = 0
    n_max = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5):
            d = random.randint(1, n - 1)
            graph = generate_d_regular_graph(n, d)
            if not is_valid_quiver_representation(graph):
                continue

            adj_list = graph
            m_index = compute_minimal_index_of_automorphism_groups(adj_list)
            proof_steps = construct_frege_proof(adj_list)
            w_F = compute_frege_proof_depth(proof_steps)

            instances_tested += 1
            n_max = max(n_max, n)

            if m_index > 2 * w_F:
                conjecture_holds = False
                counterexample = f"n={n}, d={d}, m_index(G)={m_index}, w_F( $\phi_G$ )={w_F}"
                break

        if not conjecture_holds:
            break

    return {
        "metric_name": metric_name,
        "metric_value": total_metric_value / instances_tested if instances_tested > 0 else 0,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unreachable")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_200fed65.py", line 142, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_200fed65.py", line 108, in run_trial
    if not is_valid_quiver_representation(graph):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_200fed65.py", line 44, in is_valid_quiver_representation
    if len(graph[i]) != d:
                   ^
NameError: name 'd' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'd', which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code, then rerun the test with a valid input to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16626
2	propose	ollama_remote	glm4:latest	0	0	12118
3	propose	ollama_remote	glm4:latest	0	0	12444
4	preregistration	ollama_remote	glm4:latest	0	0	10406
5	novelty	ollama_remote	glm4:latest	0	0	8827
6	novelty	ollama_remote	glm4:latest	0	0	9904

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16435
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20254
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19775
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13919
11	judge	ollama_remote	glm4:latest	0	0	23197

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 163904 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/6551baf88517.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/6551baf88517.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/6551baf88517.tar.gz* (if generated)